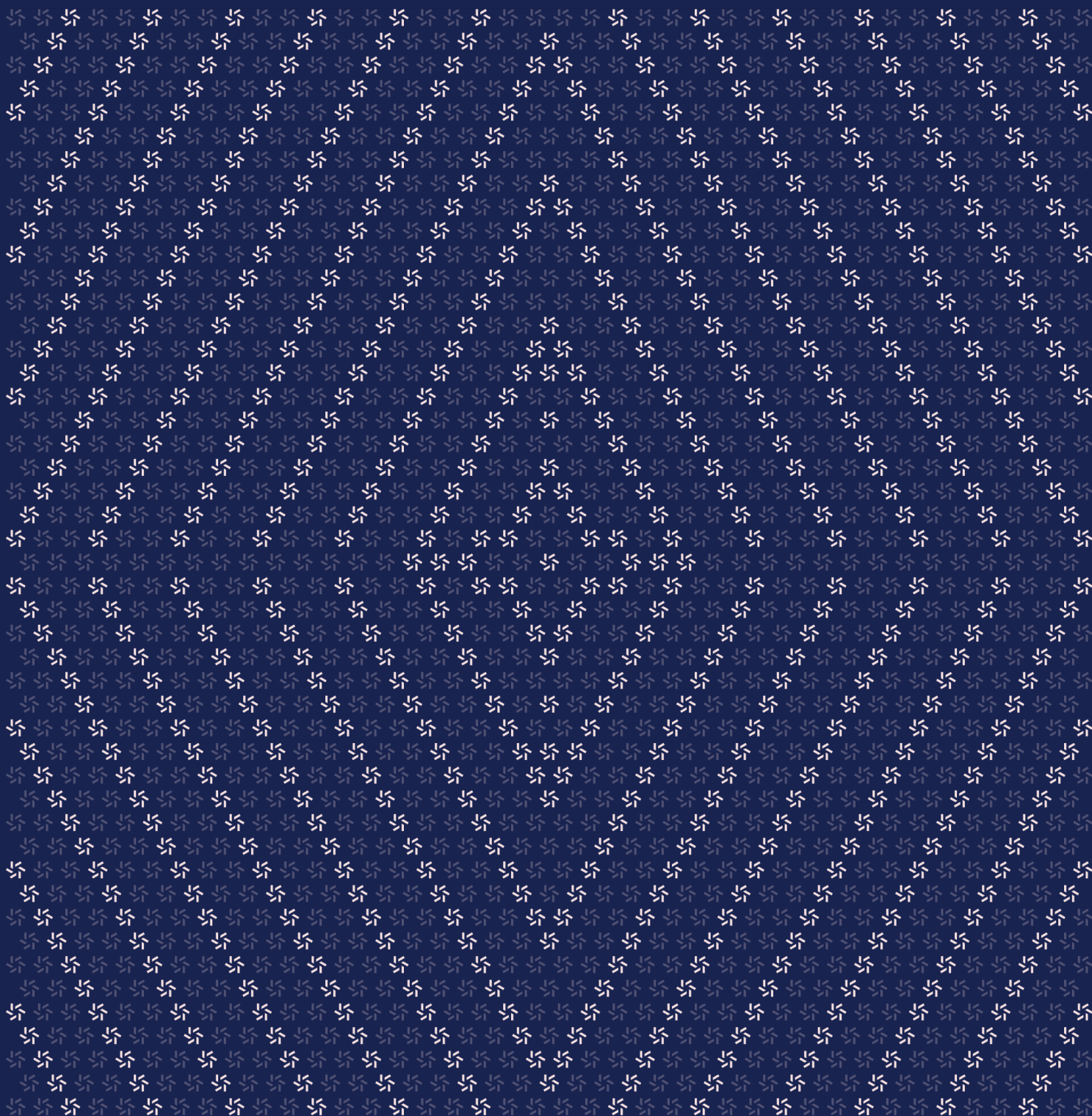


January 12, 2026

FortyTwo Prediction Market Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About FortyTwo Prediction Market	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. The excessive redeem tax for a large redeem amount can be bypassed	11
3.2. The function <code>clamp</code> does not validate that the parameter <code>lower</code> is less than the parameter <code>upper</code>	13
3.3. Lack of validation in <code>setTreasury</code> function	14
3.4. Insufficient test coverage	15
3.5. Lack of validation	17
3.6. Unnecessary <code>payable</code> modifier in the function <code>multicall</code>	19
3.7. The <code>guess0tDelta</code> functions contain repeated calculations and can be optimized	21

3.8.	The function <code>mintCollateralToExact0t</code> does not check if the receiver is a market	24
3.9.	The <code>setProtocolFeeRate</code> function does not allow to set fee up from zero	26
3.10.	String slice compare fails to mask final partial word	27
3.11.	UTF-8 decoding accepts invalid four-byte-leading bytes and produces out-of-range code points	29
4.	Discussion	29
4.1.	Unused function parameters	30
4.2.	Inconsistency between the implementation and comments in the function <code>resolve</code>	30
4.3.	Add sanity checks before calculating the amount of outcome tokens to save gas	31
4.4.	Unnecessary approval in function <code>_executeRedeemExact0tToCollateral</code>	32
5.	Threat Model	33
5.1.	Module: <code>ActionSimple.sol</code>	34
5.2.	Module: <code>FTMarket.sol</code>	39
5.3.	Module: <code>FTMarketController.sol</code>	51
5.4.	Module: <code>FTRouter.sol</code>	67
5.5.	Module: <code>Multicallbackable.sol</code>	68
6.	Assessment Results	69
6.1.	Disclaimer	70

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for FortyTwo from December 18th, 2025 to January 12th, 2026. During this engagement, Zellic reviewed FortyTwo Prediction Market's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there any vulnerabilities that could result in the loss of user funds?
 - Have important functions implemented proper access control?
 - Are there any vulnerabilities that could allow draining the market?
 - Are there safer alternatives to the curve and redeem formulas that effectively have similar properties?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

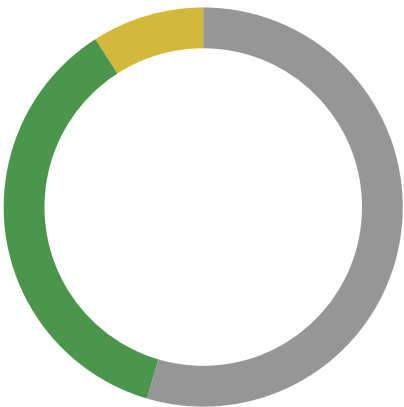
1.4. Results

During our assessment on the scoped FortyTwo Prediction Market contracts, we discovered 11 findings. No critical issues were found. One finding was of medium impact, four were of low impact, and the remaining findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of FortyTwo in the Discussion section ([4.7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	1
Low	4
Informational	6



2. Introduction

2.1. About FortyTwo Prediction Market

FortyTwo contributed the following description of FortyTwo Prediction Market:

FortyTwo is a prediction markets protocol where participants mint(buy) and redeem(sell) Outcome Tokens (OT). The marginal mint and redeem prices are determined via bonding curves. Upon the market's final resolution, holders of the winning OT can claim to receive a payout proportional to the total pool. The odds of each outcome are determined by the pool's distribution of OT (i.e. the relative supply of each OT reflect the outcome's implied probability). OTs can be purchased with ERC20s or with OTs from other markets. Markets that accept OTs as collateral are referred to as "chained" markets.

FortyTwo's design comes with characteristics worth highlighting:

- **Bonding Curve Pricing:** OT prices are set algorithmically by a bonding curve — meaning the higher the total supply, the higher the price.
- **Redemption Spread:** An early redemption spread (penalty) is applied to stabilise markets and encourage holding till resolution. The current spread calculation is a function of swap amount, outstanding supply of specified OT and time-to-maturity.
- **Profit Risk:** Holding the winning OT implies a payout. However, holding the winning OT does not guarantee a profit as you may have minted it at a price higher than payout.
- **Liquidity Bootstrapping:** Markets don't rely on external market-makers — liquidity is handled natively through the bonding curve.
- **Curve Agnostic:** A market's bonding curve can be set with unique functions and/or parameters.
- **Expandable Outcomes:** New outcomes can be introduced after a market has launched.

Lastly, FortyTwo operates a centralised resolution registry to process market resolutions.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model

checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4.7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

FortyTwo Prediction Market Contracts

Type	Solidity
Platform	EVM-compatible
Target	ft-contracts
Repository	https://github.com/fortytwo-protocol/ft-contracts
Version	99bbcf7da521b8c738eab766c534318bacd31c36
Programs	controller/mixins/*.sol controller/FTMarketController.sol FTERC6909.sol FTMarket.sol curves/PowerCurve.sol curves/CurveBase.sol curves/math/PowerMath.sol FTRouter.sol router/*.sol libraries/*.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 3.1 person-weeks. The assessment was conducted by two consultants over the course of two calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Katerina Belotskaia
✈ Engineer
kate@zellic.io ↗

Qingying Jie
✈ Engineer
qingying@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

December 18, 2025	Start of primary review period
--------------------------	--------------------------------

December 20, 2025	Kick-off call
--------------------------	---------------

January 12, 2026	End of primary review period
-------------------------	------------------------------

3. Detailed Findings

3.1. The excessive redeem tax for a large redeem amount can be bypassed

Target	FTMarket		
Category	Business Logic	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

The redeem tax is necessary to discourage large redemptions that could rapidly drain pool collateral. The tax rate is clamped between constants `MINIMUM_TAX_RATE` and `MAXIMUM_TAX_RATE`, which are set to 0.1% and 100%, respectively. It is calculated using `growthFactor`, `supplyFactor`, and `timeFactor` in the `calRedeemTaxRate` function.

```
function calRedeemTaxRate(RedeemParams memory self, uint256 otSupply,
    uint256 otDelta)
    internal
    pure
    returns (uint256)
{
    //r = min(1,growthfactor*supplyfactor*timefactor)
    uint256 growthFactor = self.calGrowthFactor(otSupply, otDelta);
    uint256 supplyFactor = calSupplyFactor(otSupply);
    uint256 timeFactor = self.calTimeFactor();
    uint256 rate = growthFactor.fullMulDivUp(supplyFactor,
        FTMath.FT_ONE).fullMulDivUp(timeFactor, FTMath.FT_ONE);

    return FTMath.clamp(rate, MINIMUM_TAX_RATE, MAXIMUM_TAX_RATE);
}
```

The time factor affects the resulting rate only if the time passed is greater than the `timeKink` constant. Before `timeKink`, the time factor is 1. After the `timeKink`, the time factor multiplicatively increases the tax rate. The `supplyFactor` depends on the current `otSupply` and is set to 1 if the `otSupply` is less than or equal to one million, 1.25 if the `otSupply` is between one million and 10 million, and 1.5 if the `otSupply` is greater than 10 million. Also, the tax rate depends on the `growthFactor`, which is calculated based on the redeem-shares amount, and it multiplies directly into the tax rate. So a larger redemption share amount increases the tax rate exponentially.

However, since the redeem process does not imply any limitation for the minimum redeem amount, or limitation on the number of the redemptions by the same user and during the same transaction, there is an opportunity to bypass the tax rate by splitting the large redeem amount into multiple small amounts and calling the redeem function multiple times.

Impact

Users still can perform large redemptions by splitting the large amount into several small amounts, but the tax rate will be significantly lower than the tax rate for the large one-redemption attempt.

Recommendations

Consider implementing the per-address redeem accumulator or cooldowns.

Remediation

This issue has been acknowledged by FortyTwo.

FortyTwo provided the following response to this finding:

On cooldowns, it's wasn't chosen as it may affect chain markets. For e.g in $A \rightarrow B \rightarrow C1$ and $A \rightarrow B \rightarrow C2$, and selling both C1 and C2 into A, the transaction is now sequence dependent as it can fail.

For per-address accumulators, it can be on a per-block or a per-market. What we realised is that the effectiveness of a per-block basis degrades quickly as we're on chains where block times are less than a second. A per-market basis would not work well for active traders with many mints and redeems.

Conversely, for public launch, we are adjusting the parameters in such a way that the time factor plays a large role than the % sold, thus reducing the impact of splitting the trade size.

We're trying to revamp the entire redeem formula and how it works. Initially, we approached it from a theoretical perspective which did not work well onchain.

3.2. The function `clamp` does not validate that the parameter `lower` is less than the parameter `upper`

Target	FTMath		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The function `clamp` limits the value of `x` between the `lower` and `upper` bounds. If it exceeds the range, it returns the boundary value. However, it does not check whether `lower` is less than `upper`.

```
function clamp(uint256 x, uint256 lower, uint256 upper)
    internal pure returns (uint256 res) {
        res = x;
        if (x < lower) res = lower;
        else if (x > upper) res = upper;
    }
```

Impact

If the function calling the function `clamp` does not check the passed `lower` and `upper` parameters, it may result in a return value less than `lower`. For example, when `upper < x < lower`, it will return the upper value, which is less than the `lower`.

Recommendations

Consider checking if `lower` is less than `upper`.

Remediation

This issue has been acknowledged by FortyTwo, and a fix was implemented in commit [5b3e6c0d](#).

3.3. Lack of validation in setTreasury function

Target	FTMarketController		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

The setTreasury function does not validate that the new treasury address specified by the admin is not zero.

```
function setTreasury(address treasuryNew)
    external nonReentrant onlyRole(DEFAULT_ADMIN_ROLE) {
        _setTreasury(treasuryNew);
    }
```

Impact

If the treasury address is zero, minting and redeeming actions will be blocked, since the process of transferring the fee collateral to the treasury will revert.

Additionally, the claim function can be affected — but only in the limited case when there is no winner, so all collateral will be transferred to the treasury. However, since there is no fee during claiming, the normal flow will not be processed without reverting.

Recommendations

Add a validation check in the setTreasury function to ensure that the treasuryNew address is not zero.

Remediation

This issue has been acknowledged by FortyTwo, and a fix was implemented in commit [cbcd7a27](#).

3.4. Insufficient test coverage

Target	Project-wide		
Category	Protocol Risks	Severity	Low
Likelihood	N/A	Impact	Low

Description

When building a complex contract with multiple moving parts (state changes) and dependencies, comprehensive testing is essential. This includes testing for both positive and negative scenarios on every function that touches the contract's state.

Positive tests should verify that each function's side effect is as expected, while negative tests should cover every revert, preferably in every logical branch.

The test coverage for this project should be expanded to include all contracts and all functions — not just surface-level functions. It is important to test the invariants required for ensuring security and also verify any mathematical properties from the project's specification. Additionally, testing cross-chain function calls and transfers is recommended to ensure the desired functionality.

Unit test cases that are missing include (but are not necessarily limited to) those documented in section [5.7](#).

Additionally, here is a list of the test cases that could be added:

- Multistep tests that including mint, redeem and claim actions for several users, also with time skip and fee updates to assert global invariants continuously across many operations (for example, for no round-trip profit, market cap tracks internal accounting)
- Tests with a nonzero fee
- Tests with different collateral decimals (not only 18 decimals but also six and eight decimals)
- The direct test for the `PowerRedeem.guessOtDelta` function with different parameters, like different `otDeltaGuessOffchain`, `eps`, or `maxIterations`
- Positive and negative tests for the `decodeGuessParam` function, including tests when data is empty or does not match the expected format

Impact

Good test coverage has multiple effects:

- It finds bugs and design flaws early (preaudit or prerelease).

- It gives insight into areas for optimization (e.g., gas cost).
- It displays code maturity.
- It bolsters customer trust in the product.
- It improves understanding of how the code functions, integrates, and operates — for developers and auditors alike.
- It increases development velocity long-term.

The last point may seem contradictory given the time investment to create and maintain tests. However, tests help developers trust their own changes. It is difficult to know if a code refactor — or even just a small one-line fix — breaks other code if there are no tests. This is especially true for new developers or those returning to the code after a prolonged absence.

Tests have your back here. They are an excellent indicator that the existing functionality was most likely not broken by a change to the code. Without a comprehensive test suite, the above benefits are lost. This increases the likelihood of bugs and vulnerabilities going unnoticed until after deployment, which can be costly and damaging to the project's reputation.

Recommendations

We recommend expanding the test suite to ensure extensive and thorough test coverage.

Remediation

This issue has been acknowledged by FortyTwo.

3.5. Lack of validation

Target	PowerRedeem		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The `redeemExactOtToCollateral` function accepts `otDeltaIn`, which determines the amount of outcome tokens (OTs) to be redeemed. Even if the `burn` function contains the validation that the caller balance is insufficient, there is no check that the `otDeltaIn` is not greater than the total supply of the token. As a result, the underflow occurs when `totalSupply - otDeltaIn`, and it causes a panic revert in the `PowerRedeem.calSwap` function.

Impact

Missing input validation may allow a user-provided value to trigger a panic revert.

Recommendations

Add a check to validate that the delta is not greater than the total supply of the token. Or add a check to `calSwap` that `otFrom` is greater than `otDelta`.

```
function calSwap(
    PowerMath.CurveParams memory curve,
    RedeemMath.RedeemParams memory redeem,
    uint256 feeRate,
    uint256 otSupply,
    uint256 otDelta
) internal pure returns (uint256 collateralToUser,
    uint256 collateralToTreasury) {
    // collateral from pool = cost * (1-taxRate)
    // fee to treasury = cost * (1-taxRate) * (feeRate)
    if (otDelta == 0) revert Errors.MarketSwapAmountCannotBeZero();

    uint256 otFrom = otSupply;
    + require(otFrom > otDelta, "OT delta is greater than total supply");
    uint256 otTo = otFrom - otDelta;
    uint256 collateralFrom = curve.calCost(otFrom);
    uint256 collateralTo = curve.calCostUp(otTo);
}
```

```
// [...]  
}
```

Remediation

This issue has been acknowledged by FortyTwo.

FortyTwo provided the following response to this finding:

It is true that it will cause an underflow panic. However, the same is the case for overflow panics for mint. Both of which prevent actual overflow and underflow vulnerabilities.

3.6. Unnecessary payable modifier in the function multicall

Target	Multicallbackable		
Category	Coding Mistakes	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The function multicall does not handle native tokens, but it has a payable modifier.

```
function multicall(Call[] calldata calls) external payable returns (Result[]
    memory returnDatas) {
    // [...]

    returnDatas = _multicall(calls);

    initiator = address(0);
}

function _multicall(Call[] memory calls) internal returns (Result[]
    memory returnDatas) {
    // [...]

    for (uint256 i = 0; i < length; ++i) {
        Call memory call = calls[i];
        (bool success, bytes memory returnData)
        = self.delegatecall(call.callData);
        // [...]
    }
}
```

Impact

If a user accidentally attaches native tokens when calling the function multicall, it will result in their native tokens being locked in the contract.

Recommendations

Consider removing the payable modifier from the function multicall.

Remediation

This issue has been acknowledged by FortyTwo, and a fix was implemented in commit [d68bc12a](#).

3.7. The guessOtDelta functions contain repeated calculations and can be optimized

Target	PowerMint, PowerRedeem		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

Both guessOtDelta functions from PowerRedeem and PowerMint contain a repeated step and can be optimized to avoid the double calculations.

If otDeltaGuessOffchain is not zero, the function goes to the first step to validate if it already satisfies the eps (approximate equality) condition. Otherwise, the function calculates the tickCurr using the marginal price for the totalSupply and goes to the second step.

During the first step of this process, tickCurr is calculated using otDeltaGuessOffchain, and after, the otDeltaGuessCurr is calculated based on the tickCurr. The otDeltaGuessCurr is used to get the collateralDelta from the calSwap function. If the difference between the target collateralDeltaTarget and the resulting collateralDelta is greater than the eps condition, the function recalculates otGuessMax and otGuessMin (tickCurr remains unchanged) and goes to the next step.

The second step in a for loop calculates the otDeltaGuessCurr using the current tickCurr; it also calls calSwap and validates the result, which is identical to the first step. After that, if the result is not satisfied, the tickCurr, otGuessMax, and otGuessMin are recalculated and the loop continues.

```
function guessOtDelta(PowerMath.CurveParams memory curve,
    RedeemMath.RedeemParams memory redeem, uint256 feeRate,
    GuessParam memory guess, uint256 collateralDeltaTarget, uint256 otSupply,
    uint256 tick
) internal pure returns ( uint256,
    /*otDeltaGuess*/ uint256 /*collateralDeltaReturned*/)
{
    // 0. save current tick pointer for refining bounds
    uint256 tickCurr;
    guess.otGuessMin = 0;
    guess.otGuessMax = 0;

    // 1. clear easiest validation
    if (guess.otDeltaGuessOffchain != 0) {
```

```
ptr.tickCurr = FTMath.clamp(guess.otDeltaGuessOffchain / tick, 1,
ptr.tickMax); // note: may not be multiplier of tick?
uint256 otDeltaGuessCurr = ptr.tickCurr * tick;

(uint256 collateralDelta,) = PowerRedeem.calSwap(curve, redeem,
feeRate, otSupply, otDeltaGuessCurr);

if (FTMath.isASmallerApproxB(collateralDelta, collateralDeltaTarget,
guess.eps)) {
    return (otDeltaGuessCurr, collateralDelta);
}
if (collateralDelta > collateralDeltaTarget) {
    // 2a. set one bound for early exit in refinement (case: overguess)
    guess.otGuessMax = tickCurr;
} else {
    // 2a. set one bound for early exit in refinement (case:
underguess)
    guess.otGuessMin = tickCurr;
}
}
// [...]

// 2b. refine both bounds via discrete newton & current tick pointer (must
use ticks, else it degrades)
for (ptr.iter = 0; ptr.iter < guess.maxIterations; ++ptr.iter) {
    uint256 otDeltaGuessCurr = ptr.tickCurr * tick;
    (uint256 collateralDelta,) = PowerRedeem.calSwap(curve, redeem,
feeRate, otSupply, otDeltaGuessCurr);

    // hit target by chance return early
    if (FTMath.isASmallerApproxB(collateralDelta, collateralDeltaTarget,
guess.eps)) {
        return (otDeltaGuessCurr, collateralDelta);
    }
}
// [...]
}
```

Impact

If the specified `otDeltaGuessOffchain` is not zero, the function will perform the same calculations twice and consume more gas than necessary.

Recommendations

Update the `tickCurr` after the first step to prepare it for the next calculations.

Remediation

This issue has been acknowledged by FortyTwo, and a fix was implemented in [PR #54](#). They intend to merge these changes into the production branch.

3.8. The function mintCollateralToExactOt does not check if the receiver is a market

Target	FTMarket		
Category	Business Logic	Severity	Informational
Likelihood	Medium	Impact	Informational

Description

The contract FTMarket overrides the `_transfer` function of the contract FTERC6909 to prevent users from initiating transfers of OTs to an FTMarket instance.

```
function _transfer(address from, address to, uint256 id, uint256 amount)
    internal override {
        FTMarketController controller = FTMarketController(factory);
        if (controller.isMarket(to) && !controller.isMarket(msg.sender)) {
            revert Errors.MarketReceiverIsMarket();
        }
        super._transfer(from, to, id, amount);
    }
```

However, the function `mintCollateralToExactOt` does not check if the receiver is an FTMarket instance when minting OTs.

```
function mintCollateralToExactOt(
    address receiver,
    // [...]
) external nonReentrant whenUnpaused returns (uint256 collateralIn) {
    MarketState memory market = readState();
    if (block.timestamp < market.timestampStart)
        revert Errors.MarketNotStarted();
    if (market.timestampEnd <= block.timestamp) revert Errors.MarketEnded();
    if (market.answer != 0 || market.isFinalised)
        revert Errors.MarketResolved();

    collateralIn = _mintCollateralToOt(market, tokenId, otDeltaOut, dataSwap,
        dataCallback, receiver);
}

function _mintCollateralToOt(
```

```
// [...]  
address receiver  
) internal returns (uint256) {  
    // [...]  
  
    _mint(receiver, tokenId, otDeltaOut);
```

Impact

If a user accidentally sets the address of an FTMarket instance as the receiver, not only will the user spend the collateral but the minted OTs will also be locked in the contract.

Recommendations

Consider adding a check in the function `mintCollateralToExactOt` to verify whether the receiver is a market.

Remediation

This issue has been acknowledged by FortyTwo, and a fix was implemented in [PR #55](#). They intend to merge these changes into the production branch.

3.9. The setProtocolFeeRate function does not allow to set fee up from zero

Target	FTMarketController		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

If, during market deployment, the fee is set to zero by the MARKET_CREATOR_ROLE, it cannot be updated to a nonzero value because the setProtocolFeeRate only allows a fee decrease.

```
function _setProtocolFeeRate(address market, uint80 feeRate) internal {
    if (feeRates[market] <= feeRate)
        revert Errors.FactoryFeeRateCanOnlyDecrease();
    _setFeeRate(market, feeRate);
}
```

Impact

After deployment of the market with zero fee, it is not possible to update the fee if it has not been set up during deployment.

Recommendations

If zero fee is allowed, but it is expected that the fee can be reset from zero to a nonzero value, consider adding a special case in the deployMarketAndSeedLiquidity function. Otherwise, consider adding a check in the deployMarketAndSeedLiquidity function to ensure that the fee is nonzero, or add a default fee value if the fee is not specified.

Remediation

This issue has been acknowledged by FortyTwo, and a fix was implemented in commit [274b9dcc](#).

FortyTwo provided the following response to this finding:

We chose the first option, although with that it does seem like it's pointless to ensure that fee decreases. We instead removed it and allow it to float freely within defined boundaries.

3.10. String slice compare fails to mask final partial word

Target	StringLib		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `StringLib.compare` function compares two slices by loading 32-byte words and applying a mask for the final partial word. The implementation only applies the mask when the entire shortest slice length is less than 32 (i.e., `shortest < 32`), rather than when the current remaining chunk is shorter than 32 bytes. When the shortest slice is greater than or equal to 32 but the final iteration contains a partial chunk (i.e., `shortest % 32 != 0`), the final loaded word is not masked. As a result, `compare` can read and include bytes beyond the declared end of each slice; if those out-of-slice bytes differ, `compare` returns a nonzero difference even though all declared bytes are identical.

Impact

Functions that rely on deterministic, canonical lexical comparison of byte slices (sorting, deduplication, equality checks, lookups keyed by strings, outcome deduplicators, registries, etc.) can behave incorrectly. Some consequences include

- incorrectly treating equal strings as different (breaking deduplication and uniqueness guarantees),
- unstable or nondeterministic ordering in sorts,
- failed equality checks, and
- subtle higher-level logic bugs where string identity is security sensitive (e.g., name deduplication, outcome-name checks).

This is not a direct memory-safety or fund-loss flaw, but it can cause state corruption, incorrect behavior, or denial of service at higher protocol layers that assume reliable slice comparisons.

Recommendations

The mask should be applied to the remaining bytes, not only when the total shortest length is less than 32.

Remediation

This issue has been acknowledged by FortyTwo, and a fix was implemented in commit [e3b68572](#).

3.11. UTF-8 decoding accepts invalid four-byte-leading bytes and produces out-of-range code points

Target	StringLib		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `StringLib.ord` function treats any leading byte greater than or equal to `0xF0` as the start of a valid four-byte UTF-8 sequence and only verifies continuation-byte bit patterns and available length. It does not enforce the canonical upper bound for four-byte sequence lead bytes ($\leq 0xF4$), nor does it validate the decoded code point range ($\leq 0x10FFFF$), exclude UTF-16 surrogate code points, or detect overlong encodings. As a result, inputs beginning with `0xF5–0xFF` followed by valid continuation bytes are accepted and decoded into numeric code points that may be outside Unicode, be in the surrogate range, or represent overlong encodings. Instead of rejecting the sequences as malformed UTF-8, `ord` returns these invalid numeric values.

Impact

Clients and higher-level protocol code that rely on `ord` for Unicode validation, canonicalization, length/count calculations, indexing, or security checks may be misled by malformed or out-of-range code points.

Recommendations

Consider enforcing canonical UTF-8 in the `ord` function.

Remediation

This issue has been acknowledged by FortyTwo, and a fix was implemented in commit [c7459a54](#).

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Unused function parameters

Some functions silence the compiler's warning about unused parameters by self-assigning values to them, such as the function `calMintCostByOtDelta` in the `PowerCurve` contract, the function `decimals` in the `FTEC6909` contract, and the function `onMint` in the `FTRouter` contract.

```
function calMintCostByOtDelta(address market, uint256 tokenId,
    uint256 otDelta, bytes calldata data)
    external
    view
    returns (uint256 collateralFromUser, uint256 collateralToTreasury)
{
    data = data; // silence compiler warning
    // [...]
}
```

Comment out the parameter name or by set the parameter to unnamed to omit specific function parameters, for example `bytes calldata /*data*/`.

Remediation

This issue has been acknowledged by FortyTwo, and a fix was implemented in commit [10b1b2a2](#).

4.2. Inconsistency between the implementation and comments in the function `resolve`

A question has a variable `timestampEnd`, which indicates the timestamp by which users can trade OTs; it also has a variable `timestampFinalise`, which records the timestamp when the question is finalized.

```
struct QuestionState {
    // [...]
    uint128 timestampEnd;
    uint128 timestampFinalise;
```

```
string[] names;
}
```

The function `resolve` in the library `Question` can update the answer before the question is finalized, but the comment says `can resolve & re-resolve before question end`. Consider making the comment consistent with the implementation.

```
function resolve(QuestionState storage self, uint256 answer) internal {
    // [...]
    // can resolve & re-resolve before question end
    // cannot re-resolve the same answer
    // cannot resolve a null-answer (outcomes are exhaustive => at least 1
    winner)
    // cannot re-resolve when it's already finalised
    if (!self.isRegistered()) revert Errors.RegistryNotRegistered();
    if (self.isFinalised()) revert Errors.RegistryAlreadyFinalised();
    if (answer == EMPTY_ANSWER || answer >= 2 ** self.names.length)
        revert Errors.RegistryInvalidAnswer();
    if (answer == self.answer) revert Errors.RegistrySameAnswer();

    // [...]
}
```

Remediation

This issue has been acknowledged by FortyTwo.

4.3. Add sanity checks before calculating the amount of outcome tokens to save gas

The functions `_mintExactCollateralToOtSimple` and `_redeemOtToExactCollateralSimple` call the functions `calOtDeltaByMintCost` or `calOtDeltaByRedeemValue` in the contract `PowerCurve` before interacting with the contract `FTMarket`. These two functions in the contract `PowerCurve` are used to calculate the corresponding OT amount based on the collateral amount provided by the user.

However, these functions do not check the validity of the provided data, such as whether the market is open or if the `tokenId` is valid. If the user provides incorrect data, the transaction may only revert when interacting with the contract `FTMarket`, by which time the calculation would have already consumed a considerable amount of gas.

Consider adding some sanity checks before calculating the OT amount.

```
function _mintExactCollateralToOtSimple(  
    address market,  
    address receiver,  
    uint256 tokenId,  
    uint256 collateralInExact,  
    uint256 otOutMin,  
    bytes calldata dataSwap,  
    bytes calldata dataGuess  
) internal returns (uint256 collateralIn, uint256 otOut) {  
    MarketDeployParams memory params  
    = IFTMarket(market).readMarketDeployParams();  
  
    (otOut, collateralIn) =  
        IFTCurve(params.curve).calOtDeltaByMintCost(market, tokenId,  
            collateralInExact, dataGuess);  
    if (otOut < otOutMin) revert Errors.RouterSlippage();  
  
    uint256 collateralInActual = _executeMintCollateralToExactOt(market,  
        receiver, tokenId, otOut, dataSwap, params);  
    if (collateralInActual != collateralIn) revert Errors.RouterDbcViolated();  
}
```

Remediation

This issue has been acknowledged by FortyTwo.

4.4. Unnecessary approval in function _executeRedeemExactOtToCollateral

Before interacting with the contract FTMarket, the function _executeRedeemExactOtToCollateral approves the market to spend OTs in the router. However, the function redeemExactOtToCollateral burns OTs directly from the router's balance without requiring approval.

Consider removing these unnecessary token approvals from the function _executeRedeemExactOtToCollateral.

```
function _executeRedeemExactOtToCollateral(  
    // [...]  
) internal returns (uint256 collateralOut) {  
    _forceApprove(market, tokenId, market, type(uint256).max);  
    collateralOut = IFTMarket(market).redeemExactOtToCollateral(receiver,
```

```
tokenId, otIn, dataSwap);  
    _forceApprove(market, tokenId, market, 0);  
}
```

Remediation

This issue has been acknowledged by FortyTwo, and a fix was implemented in commit [4cf3c318](#).

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. Module: ActionSimple.sol

Function: `claimAllSimple(address market, address receiver)`

After the market's question is finalized, holders can claim their payout by burning all the winning OTs they hold in the specified market through this function.

Inputs

- `market`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must be a market deployed by the controller.
 - **Impact:** The address of the market to interact with.
- `receiver`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must not be zero address or equal to the specified market.
 - **Impact:** The address to receive the claimed collateral.

Branches and code coverage

Intended branches

- The expected amount of winning OTs are burned.
 - ☒ Test coverage
- Nonwinning tokens are not burned.
 - ☒ Test coverage
- The receiver successfully receives the claimed collateral.
 - ☒ Test coverage

Negative behavior

- Reverts if the market is not a valid market deployed by the controller.
 - ☐ Negative test

- Reverts if the question is not finalized.
- Negative test

Function call analysis

- `IFTMarket(market).claim(receiver, tokenIds, otToBurn)`
 - **What is controllable?** `market` and `receiver`.
 - **If the return value is controllable, how is it used and how can it go wrong?** It returns the total amount of the collateral sent to the receiver.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `claimSimple(address market, address receiver, uint256[] tokenIds, uint256[] otToBurn)`

After the market's question is finalized, holders can claim part of their payout by burning some OTs they hold through this function.

Inputs

- `market`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must be a market deployed by the controller.
 - **Impact:** The address of the market to interact with.
- `receiver`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must not be zero address or equal to the specified market.
 - **Impact:** The address to receive the claimed collateral.
- `tokenIds`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must be valid OT IDs for the specified market, and the lengths of `tokenIds` and `otToBurn` must be equal.
 - **Impact:** The IDs of OTs to be burned for claiming collateral.
- `otToBurn`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** The lengths of `tokenIds` and `otToBurn` must be equal.
 - **Impact:** The amounts of the corresponding OTs to be burned for claiming collateral.

Branches and code coverage

Intended branches

- The expected amount of OTs are burned.
 - ☒ Test coverage
- The receiver successfully receives the claimed collateral.
 - ☒ Test coverage

Negative behavior

- Reverts if the market is not a valid market deployed by the controller.
 - ☐ Negative test
- Reverts if the lengths of tokenIds and otToBurn are not equal.
 - ☐ Negative test

Function call analysis

- `IFTMarket(market).claim(receiver, tokenIds, otToBurn)`
 - **What is controllable?** market, receiver, tokenIds, and otToBurn.
 - **If the return value is controllable, how is it used and how can it go wrong?** It returns the total amount of the collateral sent to the receiver.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `swapSimple(address market, address receiver, uint256 tokenId, SwapParams params, bytes dataSwap, bytes dataGuess)`

This function allows users to exchange collateral and OTs. If it is called for minting OTs, the function must be called via the function `multicall` to set the `initiator`.

Inputs

- `market`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must be a market deployed by the controller.
 - **Impact:** The address of the market to interact with.
- `receiver`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must not be equal to the specified market.

- **Impact:** The address to receive the swapped tokens.
- tokenId
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must be a valid OT ID for the specified market.
 - **Impact:** The OT ID to mint or redeem.
- params
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** Its type struct SwapParams contains the following.
 - isMint, a boolean indicating whether to mint OTs (true) or redeem them (false)
 - amount, the amount of collateral or OTs involved in the swap
 - isExactIn, a boolean indicating whether the specified amount is an exact input amount (true) or an exact output amount (false)
 - minOutOrMaxIn, the minimum output amount when isExactIn is true or the maximum input amount when isExactIn is false
- dataSwap
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** Unused.
- dataGuess
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Its length must be zero or GUESS_PARAM_LENGTH (96). If the length is GUESS_PARAM_LENGTH, the maxIterations cannot be zero and eps cannot be greater than 1e18.
 - **Impact:** It is used for guessing the swap amount when providing the exact collateral. It contains otDeltaGuessOffchain, maxIterations, and eps. The otDeltaGuessOffchain refers to the off-chain estimation of the number of OTs that can be exchanged based on the exact collateral, maxIterations refers to the maximum number of iterations allowed for the on-chain guessing algorithm, and eps refers to the maximum allowable deviation between the actual collateral required and the specified collateral amount.

Branches and code coverage

Intended branches

- A user can successfully mint OTs by providing the exact amount of collateral.
- ☒ Test coverage
 - A user can successfully mint the exact amount of OTs.

☒ Test coverage

- A user can successfully redeem OTs by requiring the exact amount of collateral.

☒ Test coverage

- A user can successfully redeem the exact amount of OTs.

☒ Test coverage

Negative behavior

- Reverts if market is not a valid market deployed by the controller.

☐ Negative test

- Reverts if slippage occurs during minting or redeeming.

☒ Negative test

Function call analysis

- `this._mintExactCollateralToOtSimple(market, receiver, tokenId, params.amount, params.minOutOrMaxIn, dataSwap, dataGuess) -> IFTCurve(params.curve).calOtDeltaByMintCost(market, tokenId, collateralInExact, dataGuess)`
 - **What is controllable?** market, tokenId, collateralInExact, and dataGuess.
 - **If the return value is controllable, how is it used and how can it go wrong?**
The returned otOut must not be less than params.minOutOrMaxIn. The returned collateralIn must be equal to the actual collateral required in the subsequent minting operation.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `this._mintExactCollateralToOtSimple(market, receiver, tokenId, params.amount, params.minOutOrMaxIn, dataSwap, dataGuess) -> this._executeMintCollateralToExactOt(market, receiver, tokenId, otOut, dataSwap, params) -> IFTMarket(market).mintCollateralToExactOt(receiver, tokenId, otOut, dataSwap, params.parentTokenId == TokenHelper.NULL_PARENT_ID ? Fedec.encodeERC20TransferFromInitiator(params.collateral, address(this)) : Fedec.encodeTransferFromInitiator(params.collateral, params.parentTokenId, address(this)))`
 - **What is controllable?** market, receiver, tokenId, and dataSwap.
 - **If the return value is controllable, how is it used and how can it go wrong?**
The return value collateralIn will be used to determine whether slippage has occurred or if the guess result is correct.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

- `this._redeemExactOtToCollateralSimple(market, receiver, tokenId, params.amount, params.minOutOrMaxIn, dataSwap) -> this._executeRedeemExactOtToCollateral(market, receiver, tokenId, otIn, dataSwap) -> IFtMarket(market).redeemExactOtToCollateral(receiver, tokenId, otIn, dataSwap)`
 - **What is controllable?** market, receiver, tokenId, and dataSwap.
 - **If the return value is controllable, how is it used and how can it go wrong?**
The return value collateralOut will be used to determine whether slippage has occurred or if the guess result is correct.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `this._redeemOtToExactCollateralSimple(market, receiver, tokenId, params.amount, params.minOutOrMaxIn, dataSwap, dataGuess) -> IFtCurve(params.curve).calOtDeltaByRedeemValue(market, tokenId, collateralOutExact, dataGuess)`
 - **What is controllable?** market, tokenId, collateralOutExact, and dataGuess.
 - **If the return value is controllable, how is it used and how can it go wrong?**
The returned otIn must not exceed params.minOutOrMaxIn. The returned collateralOut must be equal to the actual collateral received in the subsequent redeeming operation.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

5.2. Module: FTMarket.sol

Function: `claim(address receiver, uint256[] tokenIds, uint256[] otToBurn)`

This function allows claiming the winning OTs. It can be called only after the market is resolved and finalized.

Inputs

- receiver
 - **Control:** Full control by the caller.
 - **Constraints:** Cannot be the market address or zero address.
 - **Impact:** Address to transfer the payout to.
- tokenIds
 - **Control:** Full control by the caller.
 - **Constraints:** Cannot be empty and should contain only valid token IDs. The

caller should own all the token IDs. The length of the tokenIds is equal to the length of the otToBurn array.

- **Impact:** Token IDs to be burned.
- otToBurn
 - **Control:** Full control by the caller.
 - **Constraints:** The caller should own these OT amounts.
 - **Impact:** The amount of OTs to claim.

Branches and code coverage

Intended branches

- The tokenIds contains the duplicated tokenIds and the full amount of the OTs divided between elements in the otToBurn array; as a result, the sum of these amounts is equal to the total owned amount by the caller.
 - ☒ Test coverage
- The expected payout has been transferred to the receiver.
 - ☒ Test coverage
- OTs have been burned from the caller.
 - ☒ Test coverage
- The TokenIds contains the winning and nonwinning tokenIds. Nonwinning tokenIds do not have any payout but will be burned.
 - ☒ Test coverage
- There is no winner, so all collateral will be transferred to the treasury.
 - ☒ Test coverage
- Claim is allowed during pause.
 - ☐ Test coverage
- otToBurn is zero.
 - ☒ Test coverage

Negative behavior

- The caller does not own enough amount of the tokenId.
 - ☒ Negative test
- The second call after a successful claim will revert.
 - ☒ Negative test
- The market is not finalized yet.
 - ☒ Negative test

Function call analysis

- `this._calWinningOtSupply(market) -> Market.toTokenId(i)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the ID of the token using the number of the outcome.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- `this._calWinningOtSupply(market) -> Market.isWinner(market.answer, tokenId)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns if the tokenId is a winner or not.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- `this._claim(market, otSupplyWinning, tokenIds, otToBurn, receiver) -> Market.claim(market, tokenIds, otToBurn, otSupplyWinning)`
 - **What is controllable?** tokenIds and otToBurn.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no check in this function that the tokenIds does not contain duplicated tokenIds, so the resulting payout amount can be overestimated using the same tokenId multiple times. However, further during the burning process, the second attempt to burn the same tokenId will revert, so there is no risk to exploit this.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the tokenIds length is not equal to the otToBurn length or if the tokenIds is empty.
- `this._claim(market, otSupplyWinning, tokenIds, otToBurn, receiver) -> _burn(msg.sender, tokenId, otBurned)`
 - **What is controllable?** tokenId and otBurned.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the caller balance of the tokenId is less than the otBurned amount.
- `this._transferOut(this.collateral, this.parentTokenId, market.treasury, excess) -> SafeERC20.safeTransfer(IERC20(collateralOrParentOt), to, amount)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the treasury address is zero address or the market does not own

enough collateral.

- `this._transferOut(collateral, parentTokenId, receiver, payout) -> SafeERC20.safeTransfer(IERC20(collateralOrParentOt), to, amount)`
 - **What is controllable?** `to`.
 - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if the receiver is zero address or the market does not own enough collateral.

Function: `mintCollateralToExactOt(address receiver, uint256 tokenId, uint256 otDeltaOut, bytes dataSwap, bytes dataCallback)`

This function is used to mint OTs to the receiver. The caller should provide the calculated amount of the collateral tokens. The price depends on the supply of the token and the curve parameters and grows as the supply increases, so early users pay less collateral to get the same amount of OTs than late users.

This function is available only when the market is not finalized, resolved, or ended. Also, before the market starts, users cannot mint OTs.

Inputs

- `receiver`
 - **Control:** Full control by the caller.
 - **Constraints:** Cannot be zero address.
 - **Impact:** Address to mint the OTs to.
- `tokenId`
 - **Control:** Full control by the caller.
 - **Constraints:** Should be a valid token ID (power of 2 and not zero) and should not exceed the number of outcomes.
 - **Impact:** Token ID to be minted.
- `otDeltaOut`
 - **Control:** Full control by the caller.
 - **Constraints:** Cannot be zero and should be on the tick of the curve.
 - **Impact:** Amount of OTs to be minted.
- `dataSwap`
 - **Control:** Full control by the caller.
 - **Constraints:** There are no constraints.
 - **Impact:** Additional data to be used by the curve. It is not used in the current

power curve implementation.

- dataCallback
 - **Control:** Full control by the caller.
 - **Constraints:** There are no constraints.
 - **Impact:** Additional data to be passed back to the caller via onMint callback.

Branches and code coverage

Intended branches

- The expected amount of OTs has been minted.
 - ☒ Test coverage
- The expected amount of collateral has been transferred to the market.
 - ☒ Test coverage

Negative behavior

- The caller does not have enough collateral.
 - ☐ Negative test
- The market has ended.
 - ☒ Negative test
- The market has been resolved.
 - ☒ Negative test
- The market has not started yet.
 - ☒ Negative test
- The receiver is the market address.
 - ☒ Negative test
- The token ID is not valid.
 - ☒ Negative test
- The OT delta out is zero.
 - ☒ Negative test
- The OT delta out is not on the tick of the curve.
 - ☒ Negative test
- Minting is not allowed during pause.
 - ☒ Negative test
- The receiver is zero address.

☑ Negative test

Function call analysis

- `this._mintCollateralToOt(market, tokenId, otDeltaOut, dataSwap, dataCallback, receiver) -> Market.mintCollateralToOt(market, tokenId, otDeltaOut, dataSwap)`
 - **What is controllable?** `tokenId, otDeltaOut, and dataSwap.`
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the collateral amount required to mint the specified amount of OTs. Also returns the collateral amount to be transferred to the treasury as a fee. If any of the calculations have been done incorrectly, these amounts can be overestimated or underestimated. There is no check that the inputted collateral amount is not greater than the maximum expected collateral amount, so the case of overestimation cannot be rejected. However, the caller can use the router contract, which provides slippage protection.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There is no reentrancy risk here. Also reverts if the token ID is not valid, OT delta out is zero, or OT delta out is not on the tick of the curve.
- `this._mintCollateralToOt(market, tokenId, otDeltaOut, dataSwap, dataCallback, receiver) -> IFTMintCallback(msg.sender).onMint(collateralDeltaIn, otDeltaOut, dataCallback)`
 - **What is controllable?** `otDeltaOut and dataCallback.`
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Since this is an unsafe external call, the caller gets full control over the callback. Reentrancy to this contract is not possible since the ReentrancyGuard is used. Additionally, the caller can freely use the already minted OTs. However, the payment after the callback is not possible to avoid; otherwise, the full transaction will be reverted, so the full payment is guaranteed.
- `this._mintCollateralToOt(market, tokenId, otDeltaOut, dataSwap, dataCallback, receiver) -> this._transferIn(this.collateral, this.parentTokenId, msg.sender, collateralDeltaIn) -> SafeERC20.safeTransferFrom(IERC20(collateralOrParentOt), from, address(this), amount)`
 - **What is controllable?** `N/A.`
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Can revert if the allowance is not enough or if the balance of the sender is not

enough to transfer the collateral to the market.

- `this._mintCollateralToOt(market, tokenId, otDeltaOut, dataSwap, dataCallback, receiver) -> this._transferOut(this.collateral, this.parentTokenId, market.treasury, collateralToTreasury) -> SafeERC20.safeTransfer(IERC20(collateralOrParentOt), to, amount)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Can revert if the balance of the market is not enough to transfer the collateral to the treasury or if the treasury address is zero address.
- `this._mintCollateralToOt(market, tokenId, otDeltaOut, dataSwap, dataCallback, receiver) -> market.curve.callRedeemValueByOtDelta(market.market, tokenId, otDeltaOut, dataSwap)`
 - **What is controllable?** `tokenId, otDeltaOut, and dataSwap`.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns the result of redeeming the same amount of the OTs to compare that the collateral amount does not exceed the provided collateral. So a user should never redeem more than the inputted collateral.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if the delta is not on the tick, but since the delta is the same, this check has already been passed during minting.

Function: `name(uint256 tokenId)`

This function overrides the ERC6909Metadata interface to return the name of the token. The name is constructed from the prefix "OT", the outcome name, and the collateral name.

Inputs

- `tokenId`
 - **Control:** Full control by the caller.
 - **Constraints:** Should be a valid token ID.
 - **Impact:** Token ID to get the name of.

Branches and code coverage

Intended branches

- The name is constructed correctly.

- ☒ Test coverage
 - The name is constructed incorrectly for the chained market.
- ☒ Test coverage

Negative behavior

- Invalid token ID.
- ☒ Negative test

Function: `redeemExactOtToCollateral(address receiver, uint256 tokenId, uint256 otDeltaIn, bytes dataSwap)`

This function allows redeeming the exact amount of OTs. The collateral will be transferred to the receiver. Redeeming is available only when the market is not finalized, resolved, or ended and only after the start of the market. The caller will pay a fee to the treasury, and also part of the collateral will be removed as redeem tax. This tax rate depends on the redeem amount; the bigger the amount, the higher the tax rate, and after the kink time, the tax rate will be increased as well.

Inputs

- `receiver`
 - **Control:** Full control by the caller.
 - **Constraints:** Cannot be the market address or zero address.
 - **Impact:** Address to transfer the collateral to.
- `tokenId`
 - **Control:** Full control by the caller.
 - **Constraints:** Should be a valid token ID (power of 2 and not zero) and should not exceed the number of outcomes. The caller should own this token ID.
 - **Impact:** Token ID to be redeemed.
- `otDeltaIn`
 - **Control:** Full control by the caller.
 - **Constraints:** Cannot be zero and should be on the tick of the curve. The caller should own enough amount of the `tokenId`.
 - **Impact:** Amount of OTs to be redeemed.
- `dataSwap`
 - **Control:** Full control by the caller.
 - **Constraints:** There are no constraints.
 - **Impact:** Additional data to be used by the curve. It is not used in the current power curve implementation.

Branches and code coverage

Intended branches

- The expected amount of collateral has been transferred to the receiver.
☒ Test coverage
- The OTs with a corresponding token ID have been burned from the caller.
☒ Test coverage

Negative behavior

- The market has ended.
☒ Negative test
- The market has been resolved.
☒ Negative test
- The market has not started yet.
☒ Negative test
- The receiver is the market address.
☒ Negative test
- The token ID is not valid.
☒ Negative test
- The OT delta in is zero.
☒ Negative test
- The OT delta in is not on the tick of the curve.
☒ Negative test
- The caller does not own enough amount of the tokenId.
☐ Negative test
- The receiver is zero address.
☒ Negative test
- Redeem is not allowed during pause.
☐ Negative test

Function call analysis

- `this._redeemOtToCollateral(market, tokenId, otDeltaIn, dataSwap, receiver) -> Market.redeemOtToCollateral(market, tokenId, otDeltaIn, dataSwap)`

- **What is controllable?** tokenId, otDeltaIn, and dataSwap.
- **If the return value is controllable, how is it used and how can it go wrong?**
Returns the calculated amount of the collateral to be transferred to the receiver in exchange for the provided OTs. Also returns the collateral amount to be transferred to the treasury as a fee.
- **What happens if it reverts, reenters or does other unusual control flow?**
This contract uses ReentrancyGuard. Also, unlike the minting functionality, there is no the callback execution, so there is no risk of reentrancy here.
- `this._redeemOtToCollateral(market, tokenId, otDeltaIn, dataSwap, receiver) -> this._transferOut(this.collateral, this.parentTokenId, receiver, collateralToUser) -> SafeERC20.safeTransfer(IERC20(collateralOrParentOt), to, amount)`
 - **What is controllable?** receiver.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the receiver is zero address or the market does not own enough collateral.
- `this._redeemOtToCollateral(market, tokenId, otDeltaIn, dataSwap, receiver) -> market.curve.calMintCostByOtDelta(market.market, tokenId, otDeltaIn, dataSwap)`
 - **What is controllable?** tokenId, otDeltaIn, and dataSwap.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the current price of the same amount of the OTs, which have been redeemed to check that the redeeming price is not higher than the current minting price.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the delta is not on the tick, but since the delta is the same, this check has already been passed during redeeming.

Function: seed(uint256 tokenId, uint256 otSeed, bytes dataSwap)

This function is available only for the factory. It is called during market deployment to seed the market with the initial liquidity. The factory implementation as of the time of writing does not allow using this function during the other stages of the market life cycle. The market should not be finalized, resolved, or ended.

Inputs

- tokenId
- **Control:** Full control by the caller.

- **Constraints:** Should be a valid token ID (power of 2 and not zero) and should not exceed the number of outcomes.
 - **Impact:** Token ID to be minted.
 - otSeed
 - **Control:** Full control by the caller.
 - **Constraints:** Cannot be zero and should be on the tick of the curve.
 - **Impact:** Amount of OTs to be minted.
 - dataSwap
 - **Control:** Full control by the caller.
 - **Constraints:** There are no constraints.
 - **Impact:** Additional data to be used by the curve. It is not used in the current power curve implementation.

Branches and code coverage

Intended branches

- The expected amount of OTs has been minted.
- ☒ Test coverage

Negative behavior

- Caller is not the factory.
- ☒ Negative test

Function call analysis

- `this._mintCollateralToOt(market, tokenId, otSeed, dataSwap, FTMarket.EMPTY_BYTES, market.treasury) -> Market.mintCollateralToOt(market, tokenId, otDeltaOut, dataSwap)`
 - **What is controllable?** tokenId, otSeed, and dataSwap.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns the collateral amount required to mint the specified amount of OTs. Also returns the collateral amount to be transferred to the treasury as a fee. If any of the calculations have been done incorrectly, these amounts can be overestimated or underestimated. But since there are no checks for the maximum expected collateral amount, the case of overestimation cannot be rejected. However, since this function can be executed only after the market is created, an unexpected result of the calculations is unlikely, since the market is in the initial state. But to avoid unexpected token price it is not recommended to use this function for the curve with untested parameters.

- **What happens if it reverts, reenters or does other unusual control flow?**
There is no reentrancy risk here. Also reverts if the token ID is not valid, OT seed is zero, or OT seed is not on the tick of the curve.
- `this._mintCollateralToOt(market, tokenId, otSeed, dataSwap, FTMarket.EMPTY_BYTES, market.treasury) -> _mint(address to, uint256 id, uint256 amount) -> _update(address(0), to, id, amount)`
 - **What is controllable?** to and id.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the to address is zero or the market address. Since there is no check in the setTreasury function, the to address can be zero address.
- `this._mintCollateralToOt(market, tokenId, otSeed, dataSwap, FTMarket.EMPTY_BYTES, market.treasury) -> this._transferIn(this.collateral, this.parentTokenId, msg.sender, collateralDeltaIn) -> SafeERC20.safeTransferFrom(IERC20(collateralOrParentOt), from, address(this), amount)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Can revert if the allowance is not enough or if the balance of the sender is not enough to transfer the collateral to the market.
- `this._mintCollateralToOt(market, tokenId, otSeed, dataSwap, FTMarket.EMPTY_BYTES, market.treasury) -> this._transferOut(this.collateral, this.parentTokenId, market.treasury, collateralToTreasury) -> SafeERC20.safeTransfer(IERC20(collateralOrParentOt), to, amount)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Can revert if the balance of the market is not enough to transfer the collateral to the treasury or if the treasury address is zero address.
- `this._mintCollateralToOt(market, tokenId, otSeed, dataSwap, FTMarket.EMPTY_BYTES, market.treasury) -> market.curve.calRedeemValueByOtDelta(market.market, tokenId, otDeltaOut, dataSwap)`
 - **What is controllable?** tokenId, otSeed, and dataSwap.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the result of redeeming the same amount of the OTs to compare that

the collateral amount does not exceed the provided collateral. So a user should never redeem more than the inputted collateral.

- **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the delta is not on the tick, but since the delta is the same, this check has already been passed during minting.

Function: `symbol(uint256 tokenId)`

This function overrides the ERC6909Metadata interface to return the symbol of the token. The symbol is constructed from the prefix "OT", the outcome symbol, and the collateral symbol.

Inputs

- `tokenId`
 - **Control:** Full control by the caller.
 - **Constraints:** Should be a valid token ID.
 - **Impact:** Token ID to get the symbol of.

Branches and code coverage

Intended branches

- The symbol is constructed correctly.
 - ☒ Test coverage
- The symbol is constructed incorrectly for the chained market.
 - ☒ Test coverage

Negative behavior

- Invalid token ID.
 - ☒ Negative test

5.3. Module: FTMarketController.sol

Function: `addOutcomes(byte[32] questionId, string[] names)`

This function allows a QUESTION_CREATOR_ROLE to add new outcomes to a question. The seed liquidity is not added to the new outcomes; as a result, these tokens will initially be cheaper to mint. New outcomes can be added even if the question has already been resolved.

Inputs

- `questionId`
 - **Control:** Full control by the `QUESTION_CREATOR_ROLE`.
 - **Constraints:** Should be registered already and not finalized.
 - **Impact:** The ID of the question to be updated.
- `names`
 - **Control:** Full control by the `QUESTION_CREATOR_ROLE`.
 - **Constraints:** Should not be zero array, and the new-outcomes number should not exceed the `MAX_NUM_OUTCOMES` (255). Each name cannot be empty and cannot exceed the `MAX_NAME_LENGTH` (50). The names cannot be duplicated or added before.
 - **Impact:** The names of the new outcomes.

Branches and code coverage

Intended branches

- New outcomes have been added successfully when the question has not been finalized yet.
 - ☒ Test coverage
- New outcomes have been added successfully when the question has been resolved.
 - ☐ Test coverage

Negative behavior

- The caller is not a `QUESTION_CREATOR_ROLE`.
 - ☒ Negative test
- The question is not registered.
 - ☒ Negative test
- The question is finalized.
 - ☒ Negative test
- Names are zero array.
 - ☐ Negative test
- New-outcomes number exceeds the `MAX_NUM_OUTCOMES` (255).
 - ☐ Negative test
- Names are empty.
 - ☐ Negative test
- Names are too long.

- ☐ Negative test
- Names are duplicated in names.
- ☐ Negative test
- Names were added before.
- ☐ Negative test

Function call analysis

- `this._addOutcome(questionId, names) -> Question.addOutcomes(question, dedup, names)`
 - **What is controllable?** names.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if names have not passed the length or duplicate validation.

Function: `createQuestion(QuestionParams params, uint128 timestampEnd, string[] names)`

This function allows a QUESTION_CREATOR_ROLE to create a new question.

Inputs

- `params`
 - **Control:** Full control by the QUESTION_CREATOR_ROLE.
 - **Constraints:** Length of the title cannot be empty and cannot exceed the MAX_TITLE_LENGTH (1,000). Length of the description cannot exceed the MAX_DESCRIPTION_LENGTH (30,000) but can be empty.
 - **Impact:** Contains the title and description of the question.
- `timestampEnd`
 - **Control:** Full control by the QUESTION_CREATOR_ROLE.
 - **Constraints:** Cannot be less than the current block timestamp.
 - **Impact:** The timestamp when the question ends.
- `names`
 - **Control:** Full control by the QUESTION_CREATOR_ROLE.
 - **Constraints:** Cannot be less than two or greater than the MAX_NUM_OUTCOMES (255). The length of each name cannot be empty and cannot exceed the MAX_NAME_LENGTH (50). The names cannot be duplicated.

- **Impact:** The names of the outcomes.

Branches and code coverage

Intended branches

- A question has been created successfully.

☒ Test coverage

Negative behavior

- The caller is not a QUESTION_CREATOR_ROLE.

☒ Negative test

- The question title is empty.

☒ Negative test

- The question title is too long.

☒ Negative test

- The question description is too long.

☒ Negative test

- The question timestampEnd is in the past.

☐ Negative test

- Question names are less than 2.

☒ Negative test

- Question names are greater than the MAX_NUM_OUTCOMES (255).

☒ Negative test

- Question names are duplicated.

☒ Negative test

- Question names are too long.

☒ Negative test

- A question has been created before.

☒ Negative test

Function call analysis

```
this._createQuestion(params, timestampEnd, names) ->
Question.getId(params)
```

- **What is controllable?** params.

- **If the return value is controllable, how is it used and how can it go wrong?**
Returns the hash of the title and description.
- **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- `this._createQuestion(params, timestampEnd, names) ->`
`Question.register(question, dedup, params, timestampEnd, names)`
 - **What is controllable?** params, timestampEnd, and names.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if names have not passed the length or duplicate validation.

Function: `deployMarketAndSeedLiquidity(address collateral, uint256 parentTokenId, byte[32] questionId, address curve, uint256 otSeed, uint128 timestampStart, uint80 feeRate)`

This function allows a MARKET_CREATOR_ROLE to deploy a market and seed liquidity for every outcome.

Inputs

- `collateral`
 - **Control:** Full control by the MARKET_CREATOR_ROLE.
 - **Constraints:** Cannot be zero address. If `parentTokenId` is not `NULL_PARENT_ID`, the `collateral` must be a registered market.
 - **Impact:** Collateral for the FTMarket.
- `parentTokenId`
 - **Control:** Full control by the MARKET_CREATOR_ROLE.
 - **Constraints:** Should be a valid token ID — power of 2, not zero.
 - **Impact:** The ID of the ERC-6909 token of a parent FTMarket's OT used as this market's collateral.
- `questionId`
 - **Control:** Full control by the MARKET_CREATOR_ROLE.
 - **Constraints:** The `question.title` cannot be empty Bytes32.
 - **Impact:** The ID of the already existing question (prediction event) for which the market is being created. This question is connected with a set of outcomes, which are the OTs of this market.
- `curve`
 - **Control:** Full control by the MARKET_CREATOR_ROLE.

- **Constraints:** Cannot be zero address.
 - **Impact:** The curve used to get the minting and redeeming price for the current total supply and delta of the OTs.
- `otSeed`
 - **Control:** Full control by the `MARKET_CREATOR_ROLE`.
 - **Constraints:** Cannot be zero.
 - **Impact:** The amount of seed tokens to be minted for each OT during market deployment.
- `timestampStart`
 - **Control:** Full control by the `MARKET_CREATOR_ROLE`.
 - **Constraints:** Cannot be greater than or equal to the end timestamp of the question.
 - **Impact:** The timestamp when the market starts. Minting and redeeming of the OTs are not allowed before this timestamp. If the `timestampStart` is in the past, the `timestampStart` will be overwritten by the current block timestamp.
- `feeRate`
 - **Control:** Full control by the `MARKET_CREATOR_ROLE`.
 - **Constraints:** Cannot be greater than the `MAXIMUM_FEE_RATE` (1e18).
 - **Impact:** Determines the fee rate for minting and redeeming the OTs.

Branches and code coverage

Intended branches

- The market has been created, and liquidity has been seeded for every outcome.
 - ☒ Test coverage
- The fee rate has been set for the market.
 - ☒ Test coverage

Negative behavior

- The caller is not a `MARKET_CREATOR_ROLE`.
 - ☒ Negative test
- The OT seed is zero.
 - ☐ Negative test
- The timestamp start is greater than or equal to the end timestamp of the question.
 - ☐ Negative test
- The fee rate is greater than the `MAXIMUM_FEE_RATE` (1e18).
 - ☐ Negative test

- ParentTokenId is not zero, and collateral is not a registered market.
 - ☒ Negative test
- ParentTokenId is invalid.
 - ☒ Negative test
- ParentTokenId is valid but exceeds the maximum token ID according to the number of outcomes.
 - ☒ Negative test
- Creates the same market twice.
 - ☐ Negative test

Function call analysis

- `this.getQuestion(questionId) -> Question.getQuestionParams(state)`
 - **What is controllable?** `questionId`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns question state and params, but the state is ignored. The params contains the title and description of the question. The title is validated that it is not empty Bytes32 — that means the question is registered.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- `Question.isEmptyQuestion(question)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns true if the title is empty.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here; the function just returns true or false.
- `this._deployMarket(collateral, parentTokenId, questionId, curve, timestampStart) -> EnumerableSet.contains(this.markets, collateral)`
 - **What is controllable?** `collateral`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns true if collateral is a registered market. If not, the `_deployMarket` function will revert.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- `this._deployMarket(collateral, parentTokenId, questionId, curve, timestampStart) -> Market.isValidTokenId(parentTokenId)`
 - **What is controllable?** `parentTokenId`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns true if `parentTokenId` is a valid token ID — power of 2, not zero.

- Otherwise, it returns false, and the `_deployMarket` function will revert.
- **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
 - `this._deployMarket(collateral, parentTokenId, questionId, curve, timestampStart) ->`
`this.registry.getNumOutcomes(IFTMart(collateral).questionId())`
 - **What is controllable?** `collateral`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
the new market uses as collateral the already registered market, the function will return the number of outcomes for the `questionId` of the parent market. The `parentTokenId` should not exceed the ID of the last OT.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
 - `this._deployMarket(collateral, parentTokenId, questionId, curve, timestampStart) -> this.registry.getOutcomeEnd(questionId)`
 - **What is controllable?** `questionId`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the timestamp when the question ends.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
 - `this._deployMarket(collateral, parentTokenId, questionId, curve, timestampStart) -> FTMath.max128(startBy, FTMath.toUint128(block.timestamp))`
 - **What is controllable?** `startBy`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Choose the maximum value between `startBy` and the current block timestamp. If `startBy` is in the past, the `timestampStart` will be overwritten by the current block timestamp.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
 - `this._deployMarket(collateral, parentTokenId, questionId, curve, timestampStart) -> this._create2(params)`
 - **What is controllable?** `params`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the address of the new market.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Returns zero address if the market has been created before. The `_deployMarket` function will revert because of the check that the market cannot be zero address.
 - `this._deployMarket(collateral, parentTokenId, questionId, curve, timestampStart) -> EnumerableSet.add(this.markets, market)`

- **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns false if the market has been added before, but the returned value is ignored.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- `this._seedLiquidity(market, collateral, parentTokenId, questionId, curve, otSeed) -> this.registry.getNumOutcomes(questionId)`
 - **What is controllable?** questionId.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the number of outcomes for the questionId. This number cannot exceed the maximum number of outcomes (255).
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- `this._seedLiquidity(market, collateral, parentTokenId, questionId, curve, otSeed) -> this._forceApprove(collateral, parentTokenId, market, type(uint256).max) -> IERC6909(collateralOrParentOt).approve(spender, parentTokenId, amount)`
 - **What is controllable?** spender, parentTokenId, and amount.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the spender or owner are zero address.
- `this._seedLiquidity(market, collateral, parentTokenId, questionId, curve, otSeed) -> IFTCurve(curve).calMintCostByOtDelta(market, tokenId, otSeed, MarketFactory.EMPTY_BYTES)`
 - **What is controllable?** otSeed.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the collateral required to mint the otSeed.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Returns if otSeed is not on the tick of the curve or if otSeed is zero.
- `this._seedLiquidity(market, collateral, parentTokenId, questionId, curve, otSeed) -> this._transferFrom(collateral, parentTokenId, msg.sender, address(this), collateralRequired) -> SafeERC20.safeTransferFrom(IERC20(collateralOrParentOt), from, to, amount)`
 - **What is controllable?** collateralOrParentOt.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the allowance is not enough (but since it has been approved to the maximum before, this should not happen). Also reverts if the balance of

`msg.sender` is not enough. There is not any slippage protection here.

- `this._seedLiquidity(market, collateral, parentTokenId, questionId, curve, otSeed) -> IFTMarket(market).seed(tokenId, otSeed, MarketFactory.EMPTY_BYTES)`

- **What is controllable?** `otSeed`.
- **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
- **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the marker has been ended or resolved. The minted tokens will be transferred to the treasury.

Function: `extendOutcomeEnd(byte[32] questionId, uint128 timestampEnd)`

This function allows a `QUESTION_CREATOR_ROLE` to extend the end timestamp of a question. All markets connected with this question will be affected by this extension.

Inputs

- `questionId`
 - **Control:** Full control by the `QUESTION_CREATOR_ROLE`.
 - **Constraints:** Should be a registered question ID. Should not be finalized.
 - **Impact:** The ID of the question to be extended.
- `timestampEnd`
 - **Control:** Full control by the `QUESTION_CREATOR_ROLE`.
 - **Constraints:** Should be greater than the current end timestamp.
 - **Impact:** The new end timestamp.

Branches and code coverage

Intended branches

- The question end timestamp has been extended successfully.

☒ Test coverage

Negative behavior

- The caller is not a `QUESTION_CREATOR_ROLE`.

☒ Negative test

- The question is not registered.

☒ Negative test

- The question is finalized.
 - ☒ Negative test
- The question end timestamp is not greater than the current end timestamp.
 - ☒ Negative test

Function call analysis

- `this._extendEnd(questionId, timestampEnd) -> Question.extendEnd(question, timestampEnd)`
 - **What is controllable?** `questionId` and `timestampEnd`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the question is not registered or finalized or the end timestamp is not greater than the current end timestamp.

Function: `finaliseOutcome(byte[32] questionId, uint256 answerChallenge)`

This function allows a `QUESTION_FINALISER_ROLE` to finalize a question (set final answer challenge, which cannot be changed anymore). It makes the claim function available for the winners.

Inputs

- `questionId`
 - **Control:** Full control by the `QUESTION_FINALISER_ROLE`.
 - **Constraints:** Should be registered already, not finalized, and should be resolved.
 - **Impact:** The ID of the question to be finalized.
- `answerChallenge`
 - **Control:** Full control by the `QUESTION_FINALISER_ROLE`.
 - **Constraints:** Should match the current answer of the question.
 - **Impact:** The final answer challenge.

Branches and code coverage

Intended branches

- The question has been finalized successfully.
 - ☒ Test coverage

Negative behavior

- The caller is not a QUESTION_FINALISER_ROLE.
 - ☒ Negative test
- The question is not registered.
 - ☐ Negative test
- The question is finalized.
 - ☐ Negative test
- The question is not resolved.
 - ☐ Negative test
- The answer challenge does not match the current answer.
 - ☐ Negative test

Function call analysis

- `this._finaliseOutcome(questionId, answerChallenge) -> Question.finalise(question, answerChallenge)`
 - **What is controllable?** `questionId` and `answerChallenge`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the question is not registered, finalized already, or resolved yet.
Also reverts if the final answer does not match the current answer.

Function: `pause ( )`

This function allows a PAUSER_ROLE to pause all the markets that have been created using this contract. The pause stops minting and redeeming actions.

Branches and code coverage

Intended branches

- Pause has been set successfully.
 - ☒ Test coverage
- Minting and redeeming actions have been stopped for not-finalized markets.
 - ☒ Test coverage

Negative behavior

- Caller is not a PAUSER_ROLE.
- ☐ Negative test

Function: `resolveOutcome(byte[32] questionId, uint256 answer)`

This function allows a QUESTION_RESOLVER_ROLE to resolve a question (set the answer).

Inputs

- `questionId`
 - **Control:** Full control by the QUESTION_RESOLVER_ROLE.
 - **Constraints:** Should be registered already and not finalized.
 - **Impact:** The ID of the question to be resolved.
- `answer`
 - **Control:** Full control by the QUESTION_RESOLVER_ROLE.
 - **Constraints:** Should not be empty and should not exceed the number of outcomes.
 - **Impact:** The answer to be set.

Branches and code coverage

Intended branches

- The question has been resolved successfully.
 - ☒ Test coverage
- The question has been already resolved and has been successfully resolved again. The answer has been overwritten.
 - ☐ Test coverage

Negative behavior

- The caller is not a QUESTION_RESOLVER_ROLE.
 - ☒ Negative test
- The question is not registered.
 - ☒ Negative test
- The question is finalized.
 - ☒ Negative test
- The answer is empty.

- ☐ Negative test
- The answer is greater than the number of outcomes.
- ☒ Negative test
- The answer is the same as the current answer.
- ☐ Negative test

Function call analysis

- `this._resolveOutcome(questionId, answer) -> Question.resolve(question, answer)`
 - **What is controllable?** `questionId` and `answer`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the answer is empty, greater than the number of outcomes, or the same as the current answer.

Function: `setProtocolFeeRate(address market, uint80 feeRate)`

The `MARKET_CREATOR_ROLE` can set the protocol fee rate for a specific market. The protocol does not support the default fee rate for the markets. As a result, the fee rate must be set for each market separately.

Inputs

- `market`
 - **Control:** Full control by the `MARKET_CREATOR_ROLE`.
 - **Constraints:** No constraints — the market can be nonexistent.
 - **Impact:** The address of the market the fee rate is being set for.
- `feeRate`
 - **Control:** Full control by the `MARKET_CREATOR_ROLE`.
 - **Constraints:** Cannot be greater than or equal to the current fee rate for the market. Cannot be more than the `MAXIMUM_FEE_RATE` (1e18).
 - **Impact:** Determines the fee rate for minting and redeeming the OTs.

Branches and code coverage

Intended branches

- The fee rate has been set for the market (from zero to a nonzero value).
 - ☐ Test coverage
- The fee rate has been updated for the market (from nonzero to a nonzero value less than the current fee rate).
 - ☒ Test coverage

Negative behavior

- The caller is not a MARKET_CREATOR_ROLE.
 - ☒ Negative test
- The fee rate is greater than the current fee rate for the market.
 - ☒ Negative test
- The fee rate is equal to the current fee rate for the market.
 - ☒ Negative test
- The fee rate is more than the MAXIMUM_FEE_RATE (1e18).
 - ☐ Negative test

Function: `unpause()`

This function allows a PAUSER_ROLE to unpause all the markets that have been created using this contract. Unpause allows minting and redeeming actions.

Branches and code coverage

Intended branches

- Unpause has been set successfully.
 - ☒ Test coverage
- Minting and redeeming actions are allowed again for not-finalized markets.
 - ☒ Test coverage

Negative behavior

- The caller is not a PAUSER_ROLE.
 - ☐ Negative test

Function: `unresolveOutcome(byte[32] questionId)`

This function allows a QUESTION_RESOLVER_ROLE to unresolve a question (reset the answer).

Inputs

- `questionId`
 - **Control:** Full control by the `QUESTION_RESOLVER_ROLE`.
 - **Constraints:** Should be registered already and not finalized.
 - **Impact:** The ID of the question to be unresolved.

Branches and code coverage

Intended branches

- The question has been unresolved successfully.
 - ☒ Test coverage

Negative behavior

- The caller is not a `QUESTION_RESOLVER_ROLE`.
 - ☒ Negative test
- The question is not registered.
 - ☒ Negative test
- The question is finalized.
 - ☒ Negative test
- The question is not resolved yet.
 - ☒ Negative test

Function call analysis

- `this._unresolveOutcome(questionId) -> Question.unresolve(question)`
 - **What is controllable?** `questionId`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the question is not registered, finalized, or resolved yet.

5.4. Module: FTRouter.sol

Function: `onMint(uint256 collateralIn, uint256 otOut, bytes dataCallback)`

This is a callback function that will be called when the function `mintCollateralToExactOt` of the contract `FTMarket` is invoked by the router. It transfers the required collateral from the initiator to the contract, and this collateral will then be transferred by the function `mintCollateralToExactOt` to pay for the minting of OTs.

This function must be called by the markets deployed by the controller.

Inputs

- `collateralIn`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** The initiator must have enough balance and have approved the router to transfer this amount.
 - **Impact:** The amount of collateral required to pay.
- `otOut`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The amount of OTs minted to the receiver.
- `dataCallback`
 - **Control:** Fully controlled by the router.
 - **Constraints:** The data must be correctly formatted according to the function signature.
 - **Impact:** The data containing partial token-transfer information.

Branches and code coverage

Intended branches

- If the selector in `dataCallback` matches `erc20TransferFromInitiator.selector`, it decodes the data and transfers ERC-20 tokens from the initiator to the specified receiver.
 - ☑ Test coverage
- If the selector in `dataCallback` matches `transferFromInitiator.selector`, it decodes the data and transfers tokens (ERC-20 or ERC-6909) from the initiator to the specified receiver.
 - ☑ Test coverage

Negative behavior

- Reverts if the caller is not a market deployed by the controller.
 - ☒ Negative test
- Reverts if the `initiator` is not set.
 - ☐ Negative test
- Reverts if the `initiator` does not have enough collateral approved for transfer.
 - ☒ Negative test

Function call analysis

- `this._multicall(calls) -> self.delegatecall(call.callData)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
 - **What happens if it reverts, reenters or does other unusual control flow?** It transfers the required amount of collateral from the `initiator` and reverts if the transfer fails.

5.5. Module: Multicallbackable.sol

Function: `multicall(Call[] calls)`

This function allows an externally owned account to execute multiple router functions in a single transaction. It sets the `initiator` to the caller's address and resets the `initiator` to zero address after execution.

Inputs

- `calls`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Only functions within the contract `FTRouter` can be called.
 - **Impact:** The array of calldata to be executed. The struct `Call` contains the calldata and a boolean `allowFailure` indicating if the call is allowed to fail without reverting the entire transaction.

Branches and code coverage

Intended branches

- It can execute functions of the contract FTRouter.
- ☒ Test coverage
- The transaction will not be reverted if a call fails and its allowFailure is true.
- ☒ Test coverage

Negative behavior

- Reverts if this function is reentered.
- ☒ Negative test

Function call analysis

- `this._multicall(calls) -> self.delegatecall(call.callData)`
 - **What is controllable?** `calls`.
 - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
 - **What happens if it reverts, reenters or does other unusual control flow?** If it reverts, the variable `success` will be false, and if `call.allowFailure` is also false, the transaction will be reverted with the `returnData`. If `call.allowFailure` is true, the transaction will not be reverted and the failure will be recorded in the `returnDatas`.

6. Assessment Results

During our assessment on the scoped FortyTwo Prediction Market contracts, we discovered 11 findings. No critical issues were found. One finding was of medium impact, four were of low impact, and the remaining findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.